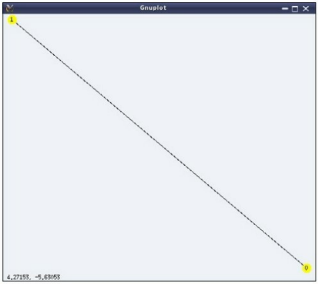

Figure 1: A simple undirected graph

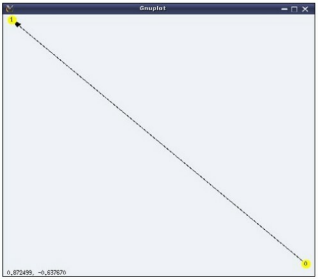

Figure 2: A simple directed graph

Note that a graph without any duplicate edges and without any loops is called a simple graph. Also, an edge from a vertex U to V is not a duplicate of an edge from vertex V to U in a directed graph but is considered a duplicate in an undirected graph. Maxima's package supports only simple graphs, i.e., graphs without duplicate edges and loops.

A simple graph can also be equivalently represented by adjacency of vertices, which means by lists of adjacent vertices for every vertex. *print_graph(<graph>)* exactly displays those lists. The following code, in continuation from the previous code snippet, demonstrates this:

```
(%i6) print_graph(g)$

Graph on 2 vertices with 1 edges.
Adjacencies:
  1 :  0
  0 :  1
(%i7) print_graph(dg)$

Digraph on 2 vertices with 1 arcs.
Adjacencies:
  1 :
  0 :  1
(%i8) quit();
```

create_graph(<num_of_vertices>, <edge_list>[, directed]) is another way of creating a graph using *create_graph()*. Here, the vertices are created as *0, 1, ..., <num_of_vertices> - 1*. So, both the above graphs could equivalently be created as follows:

```
(%i1) load(graphs)$
...
0 errors, 0 warnings
(%i2) g: create_graph(2, [[0, 1]]);
(%o2)                    GRAPH(2 vertices, 1 edges)
(%i3) dg: create_graph(2, [[0, 1]], directed=true);
(%o3)                    DIGRAPH(2 vertices, 1 arcs)
(%i4) quit();
```

make_graph(<vertices>, <predicate>[, directed]) is another interesting way of creating a graph, based on vertex connectivity conditions specified by the *<predicate>* function. *<vertices>* could be an integer or a set/list of vertices. If it is a positive integer, then the vertices would be *1, 2, ..., <vertices>*. In any case, *<predicate>* should be a function taking two arguments of the vertex type and returning true or false. *make_graph()* creates a graph with the vertices specified as above, and with the edges between the vertices, for which the *<predicate>* function returns true.

A simple case would be, if the *<predicate>* always returns true, then it would create a complete graph, i.e., a simple graph where all vertices are connected to each other. Here are a couple of demonstrations of *make_graph()*:

```
$ maxima -q
(%i1) load(graphs)$
...
0 errors, 0 warnings
(%i2) f(i, j) := true$
(%i3) g: make_graph(6, f);
(%o3)                    GRAPH(6 vertices, 15 edges)
(%i4) draw_graph(g, show_id=true, vertex_color=yellow)$
(%i5) f(i, j) := is(mod(i, j)=0)$
(%i6) g: make_graph(10, f, directed = true);
(%o6)                    DIGRAPH(10 vertices, 17 arcs)
(%i7) draw_graph(g, show_id=true, vertex_color=yellow, vertex_
size=4)$
(%i8) quit();
```